

Name = Aniket Shivilal Kasode

Roll No = SY-A-53

Sub = PDS (Practical 10)

Aim - Version Control using Git & GitHub

Theory 1. Difference between Git and GitHub

Git is a distributed version control system that allows developers to track changes in their source code during software development. It helps in maintaining a complete history of changes, managing different versions of files, and enabling collaboration among multiple developers. Git works locally on a user's computer and does not require an internet connection to perform most operations.

GitHub, on the other hand, is a web-based platform that hosts Git repositories in the cloud. It provides additional features such as collaboration tools, issue tracking, pull requests, and project management. GitHub makes it easier for teams to work together by allowing them to share code, review changes, and manage projects remotely.

In simple terms, Git is a tool, while GitHub is a service built on top of Git.

2. Difference between git add and git commit

The git add command is used to move changes from the working directory to the staging area. It allows developers to select specific files or changes that they want to include in the next commit. This step is important because it gives control over what changes are saved.

The git commit command is used to save the staged changes into the local repository. Each commit acts as a snapshot of the project at a particular point in time. It also includes a commit message that describes the changes made.

Therefore, git add prepares the changes, while git commit permanently records them in the project history.

3. What is the Staging Area in Git?

The staging area, also known as the index, is an intermediate area between the working directory and the Git repository. It is used to prepare and organize changes before committing them.

When a file is modified, it is first present in the working directory. By using the `git add` command, the file is moved to the staging area. Only the files in the staging area are included when a commit is made.

The staging area provides flexibility by allowing developers to stage only selected changes instead of committing everything at once. This helps in creating clean and meaningful commits.

4. What happens if you modify a file after committing it?

If a file is modified after it has been committed, Git will recognize the file as modified in the working directory. The previous commit remains unchanged because Git commits are immutable (cannot be altered).

The new changes are not automatically included in the previous commit. Instead, they must go through the same process again: first added to the staging area using `git add`, and then saved using `git commit`.

This ensures that Git maintains a clear and accurate history of all changes made to the project.

5. Difference between `git push` and `git pull`

The `git push` command is used to upload local commits from a developer's machine to a remote repository, such as GitHub. It is typically used when a developer wants to share their changes with others or back up their work online.

The `git pull` command is used to fetch and download changes from a remote repository to the local system. It also automatically merges those changes into the current branch.

In simple terms, `git push` sends changes to the remote repository, while `git pull` brings changes from the remote repository to the local environment.

6. What is Version Control? Differentiate between Centralized and Distributed VCS

Version Control is a system that helps in tracking and managing changes to files over time. It allows multiple users to collaborate on a project, maintain a history of modifications, and revert to previous versions if needed. It is widely used in software development to ensure efficient teamwork and code management.

There are two main types of Version Control Systems:

Centralized Version Control System (CVCS)

In a centralized system, there is a single central server that stores all the project files and their history. Users connect to this server to access or modify files. Examples include SVN (Subversion). The main disadvantage is that if the server fails, all data may be lost, and users cannot work offline.

Distributed Version Control System (DVCS)

In a distributed system, every user has a complete copy of the repository, including its history. Git is an example of a DVCS. This allows users to work offline and provides better data security because multiple copies exist. It also improves performance and flexibility in collaboration.

In conclusion, centralized systems rely on a single server, while distributed systems provide each user with a full copy of the repository, making them more robust and efficient.

SCREENSHOTS :

```

MINGW64/c/Users/anike/MyProject
anike@Aniket MINGW64 ~
$ git --version
git version 2.54.0.windows.1
anike@Aniket MINGW64 ~
$ git config --global user.name "Aniket"
anike@Aniket MINGW64 ~
$ git config --global user.email "aniketkasode0013@gmail.com"
anike@Aniket MINGW64 ~
$ mkdir MyProject
anike@Aniket MINGW64 ~
$ git init
Initialized empty Git repository in C:/Users/anike/.git/
anike@Aniket MINGW64 ~ (master)
$ mkdir MyProject
mkdir: cannot create directory 'MyProject': File exists
anike@Aniket MINGW64 ~ (master)
$ cd MyProject
anike@Aniket MINGW64 ~/MyProject (master)
$ git init
Initialized empty Git repository in C:/Users/anike/MyProject/.git/
anike@Aniket MINGW64 ~/MyProject (master)
$ echo "hello git">hello.txt
anike@Aniket MINGW64 ~/MyProject (master)
$ git add hello.txt
warning: in the working copy of 'hello.txt', LF will be replaced by CRLF the next time Git touches it
anike@Aniket MINGW64 ~/MyProject (master)
$ git commit -m "First Commit"
[master (root-commit) 42c605e] First Commit
1 file changed, 1 insertion(+)
create mode 100644 hello.txt
anike@Aniket MINGW64 ~/MyProject (master)
$ git log
commit 42c605e8406eeddaed06c92e8888a18a2a55a973 (HEAD -> master)
Author: Aniket <aniketkasode0013@gmail.com>
Date: Wed May 6 11:18:50 2026 +0530

    First Commit
anike@Aniket MINGW64 ~/MyProject (master)
$ git remote add origin https://github.com/Aniketkasode/MyProject.git

anike@Aniket MINGW64 ~/MyProject (master)
$

```

```

MINGW64/c/Users/anike/MyProject
anike@Aniket MINGW64 ~ (master)
$ cd MyProject
anike@Aniket MINGW64 ~/MyProject (master)
$ git init
Initialized empty Git repository in C:/Users/anike/MyProject/.git/
anike@Aniket MINGW64 ~/MyProject (master)
$ echo "hello git">hello.txt
anike@Aniket MINGW64 ~/MyProject (master)
$ git add hello.txt
warning: in the working copy of 'hello.txt', LF will be replaced by CRLF the next time Git touches it
anike@Aniket MINGW64 ~/MyProject (master)
$ git commit -m "First Commit"
[master (root-commit) 42c605e] First Commit
1 file changed, 1 insertion(+)
create mode 100644 hello.txt
anike@Aniket MINGW64 ~/MyProject (master)
$ git log
commit 42c605e8406eeddaed06c92e8888a18a2a55a973 (HEAD -> master)
Author: Aniket <aniketkasode0013@gmail.com>
Date: Wed May 6 11:18:50 2026 +0530

    First Commit
anike@Aniket MINGW64 ~/MyProject (master)
$ git remote add origin https://github.com/Aniketkasode/MyProject.git

anike@Aniket MINGW64 ~/MyProject (master)
$
anike@Aniket MINGW64 ~/MyProject (master)
$ git remote add origin https://github.com/Aniketkasode/MyProject.git
error: remote origin already exists.
anike@Aniket MINGW64 ~/MyProject (master)
$ git push -u origin master
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
writing objects: 100% (3/3), 224 bytes | 224.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'master' on GitHub by visiting:
remote:   https://github.com/Aniketkasode/MyProject/pull/new/master
remote:
To https://github.com/Aniketkasode/MyProject.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
anike@Aniket MINGW64 ~/MyProject (master)
$

```

XAUUSD 4.712.175 ▲ +0.53%Aniketkasode/MyProjectAsk GeminiPaused

github.com/Aniketkasode/MyProject

Aniketkasode / MyProjectType to search

<> CodeIssuesPull requestsActionsProjectsWikiSecurity and qualityInsightsSettings

MyProjectPublicPinWatchForkStar

main2 Branches0 TagsGo to fileAdd fileCode

AniketkasodeAdd files via upload2bdf602 · now2 Commits

PDS_Practical_1.ipynb	Add files via upload	now
PDS_Practical_6.ipynb	Add files via upload	now
PDS_Practical_9.ipynb	Add files via upload	now
PDS_Practical_2.ipynb	Add files via upload	now
PDS_Practical_3.ipynb	Add files via upload	now
PDS_Practical_4.ipynb	Add files via upload	now
PDS_Practical_5.ipynb	Add files via upload	now
PDS_Practical_7.ipynb	Add files via upload	now
PDS_Practical_8.ipynb	Add files via upload	now
README.md	Initial commit	2 days ago

README

MyProject

About

No description, website, or topics provided.

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors

Aniketkasode